MLOps Course: Data Pipeline Submission

Otto PM — AI-Powered Code Repository Management System

Ayushman Khandelwal, Malav Patel, Paschal Corringan,
Sahil Chute, Shloka Trivedi, Tom Howes
Group 21

February 24, 2026

Contents

1 Overview **2**

2 Key Components **2**
2.1 Data Acquisition . . . 2
2.2 Data Preprocessing . . . 3
2.3 Test Modules . . . 4
2.4 Pipeline Orchestration . . . 5
2.5 Data Versioning with DVC . . . 7
2.6 Tracking and Logging . . . 8
2.7 Data Schema & Statistics Generation . . . 8
2.8 Anomaly Detection & Alerts . . . 9
2.9 Pipeline Flow Optimization . . . 10

3 Data Bias Detection Using Data Slicing **11**
3.1 Detecting Bias in the Data . . . 11
3.2 Data Slicing for Bias Analysis . . . 12
3.3 Mitigation of Bias . . . 12
3.4 Bias Mitigation Documentation . . . 13

4 Additional Guidelines **13**
4.1 Folder Structure . . . 13
4.2 README Documentation . . . 15
4.3 Reproducibility . . . 15
4.4 Code Style . . . 16
4.5 Error Handling & Logging . . . 16

5 Evaluation Criteria Summary **17**

1 Overview

Otto is an AI-powered code repository management system with RAG (Retrieval-Augmented Generation) capabilities. The data pipeline ingests GitHub repositories, semantically chunks and embeds code, validates data quality, detects anomalies, and performs bias analysis across data slices — enabling intelligent Q&A, documentation generation, and code editing over any codebase.

The pipeline is orchestrated using Airflow for DAG scheduling and visualization, and DVC (Data Version Control) for data versioning. The pipeline wraps the same production classes used by Otto's `ingest-service` deployed on Google Cloud Run (`us-east1`), with no code duplication. All pipeline stages share the same GCP infrastructure: Google Cloud Storage for data, Vertex AI (`text-embedding-004`) for embeddings, and Firestore for user and session management.

2 Key Components

2.1 Data Acquisition

Implementation:
`ingest-service/src/ingestion/github_ingester.py`

The `GitHubIngester` class fetches entire GitHub repositories via the GitHub REST API. Key features include:

- Authenticates via GitHub personal access token or GitHub App installation token (15,000 requests/hour vs. 5,000 for PAT)
- Fetches full repository tree recursively:
  `GET /repos/{owner}/{repo}/git/trees/{branch}?recursive=1`
- Filters code files by extension: `.py .ts .tsx .js .jsx .java .cpp .c .h .go .rs .rb .php .swift .kt .scala .sql .md .yaml .yml .json .html .css .scss`
- Excludes non-code paths: `node_modules`, `__pycache__`, `.git`, `dist`, `build`, `.next`, `venv`, `coverage`, `target`
- Implements retry logic with exponential backoff: 3 retries, backoff factor 1s, retries on HTTP 429/500/502/503/504
- Decodes file content with UTF-8 fallback (`errors='ignore'`) and skips binary files (null byte detection)
- Stores raw files to GCS:
  `gs://otto-pm-raw-repos/repos/{owner}/{repo}/{file_path}`
- Tracks commit SHA via `CommitTracker` — skips re-ingestion if the repository has not changed (smart incremental updates)
- Saves `metadata.json` with: file count, commit SHA, author, commit message, language, stars, description, and ingestion timestamp
- Supports any public or private GitHub repository via the `OTTO_REPO` environment variable

External Dependencies:

```
PyGithub==2.1.1
google-cloud-storage>=3.0.0
requests
```

2.2 Data Preprocessing

Implementation:
`ingest-service/src/chunking/enhanced_chunker.py`,
`ingest-service/src/chunking/chunker.py`,
`ingest-service/src/chunking/embedder.py`

The `EnhancedCodeChunker` class performs semantic code chunking using Tree-sitter AST parsing:

Chunking Strategy:

- Chunk size: 150 lines per chunk, 10-line overlap between consecutive chunks
- Semantic chunking: identifies function and class boundaries via AST node traversal (Tree-sitter grammar per language)
- Falls back to smart line chunking for unsupported languages, detecting natural break points (empty lines, closing braces, function/class declarations)

Language-Specific Metadata Extraction:

Python:

- Imports (`import` + `from...import`, first 200 lines, deduplicated)
- Decorators (`@property`, `@staticmethod`, `@router.get`, etc.)
- Type hints (parameter types and return types per function)
- Docstrings (first 300 characters per function/class)
- Async function names
- Exception handling (raised, caught, custom exception classes)
- Class methods (instance, static, classmethod, properties)
- Global variables and constants (UPPER_CASE prioritized)

TypeScript:

- Interfaces with property names and types (up to 10 per interface)
- Type definitions (type aliases)
- Enums with values
- Generic type parameters
- Named and default exports

JavaScript:

- ESM imports (`import...from`) and CommonJS (`require()`)
- Named exports (`export const/function/class`)
- Default exports
- Async functions (3 detection patterns)

Java:

- Annotations (`@Override`, `@Autowired`, etc.)
- Interface names
- Access modifier counts (public / private / protected / static / final)

Enriched Content: Each chunk's embedding is generated from an enriched string combining: repository name + description + primary language + file path + imports + classes + functions + language-specific metadata + chunk name/type/line range + raw code content. This enrichment significantly improves RAG retrieval quality by providing full context beyond raw code alone.

Output:
`gs://otto-pm-processed-chunks/repos/{owner}/{repo}/chunks.jsonl`

Each chunk record contains: `content`, `enriched_content`, `language`, `file_path`, `chunk_index`, `chunk_type`, `start_line`, `end_line`, `num_lines`, `name`, `summary`, `embedding` (768-dim), `embedding_model`, `embedding_dim`.

External Dependencies:

```
tree-sitter==0.20.4
tree-sitter-languages==1.10.2
```

2.3 Test Modules

Implementation:
`Data-Pipeline/tests/`

Unit tests written using `pytest` covering all three core pipeline components. All GCP clients are patched using `unittest.mock` so no real credentials are needed.

Test Results: 69/69 tests passing

`test_acquisition.py` — **20 tests (`GitHubIngester`):**

- `_parse_repo_url`: full URL, `.git` suffix, short `owner/repo` format, trailing slash
- `_detect_language`: Python, TypeScript, JavaScript, Java, unknown extension, no extension
- `_filter_code_files`: excludes `node_modules`, `__pycache__`, directories, unsupported extensions; includes `.py` and `.tsx`; handles mixed trees
- Session & constructor: retry session creation, no-token behavior, token auth header format

`test_preprocessing.py` — **32 tests (`EnhancedCodeChunker`):**

- Python imports: simple, `from` style, multi-import, comment skipping, empty input, deduplication
- Python decorators: basic, with arguments, empty
- Python async: detection, sync exclusion, empty
- Python exceptions: raised, caught, custom classes, empty
- Python globals: constants, ordering priority, import skipping
- TypeScript: interfaces, type definitions, enums, generics
- JavaScript: ESM imports, CommonJS require, deduplication, relative import skipping, named exports, default exports
- Java: annotations, interfaces, access modifiers

`test_embedder.py` — **17 tests (`ChunkEmbedder`):**

- Initialization: default location, batch size (250), max text length (3072), not initialized at start
- `initialize_model`: success, failure, idempotency
- `_load_chunks`: correct parsing, empty line skipping
- `_save_chunks`: correct serialization and upload
- `embed_repository`: skips already-embedded chunks, force re-embed clears embeddings, no model returns failed
- `_generate_embeddings_batch`: truncates long text, uses enriched content, assigns metadata, handles API failure

Run tests:

```
cd Data-Pipeline
pytest tests/ -v
```

2.4 Pipeline Orchestration

Implementation:
`Data-Pipeline/dvc.yaml`,
`Data-Pipeline/dags/airflow_dag.py`,
`Data-Pipeline/scripts/run_pipeline.py`

Dual Orchestration Architecture

Otto uses a dual orchestration approach: **Airflow** for DAG scheduling, visualization, and monitoring, and **DVC** for data versioning and reproducibility. Both orchestrators call the same `run_pipeline.py` script, which imports Otto's production classes directly — there is zero code duplication between the two layers.

DAG Structure

The pipeline consists of a four-stage DAG with explicit dependencies ensuring logical execution order:

```
ingest -> chunk -> embed -> validate
```

Each stage declares its command, source-code dependencies, and data outputs. The dependency graph determines which stages require re-execution:

Stage	Description	Depends On	Output(s)
`ingest`	Downloads repo files from GitHub to GCS via `GitHubIngester`	(entry point)	• `data/raw/metadata.json`
`chunk`	Parses code into chunks via Tree-sitter AST using `EnhancedCodeChunker`	`data/raw/metadata.json`	• `data/processed/chunks.jsonl`
`embed`	Generates 768-dim embeddings via Vertex AI (`text-embedding-004`)	`data/processed/chunks.jsonl`	• `data/processed/chunks_embedded.jsonl`
`validate`	Runs schema validation, anomaly detection, and bias analysis via `ingest-service/src/validation/schema_validation.py`, `ingest-service/src/validation/anomaly_detection.py`, `ingest-service/src/validation/bias_detection.py`	`data/processed/chunks_embedded.jsonl`	• `data/processed/schema_validation.json` • `data/processed/anomaly_detection.json` • `data/processed/bias_detection.json` • `data/processed/validation_report.json` • `data/logs/pipeline.log`

Table 1: Pipeline DAG stages, dependencies, and outputs

Airflow DAG

The Airflow DAG (`dags/otto_pipeline_dag.py`) orchestrates the four stages using `BashOperator` tasks, each calling `python scripts/run_pipeline.py <stage>`. The DAG is configured with `schedule_interval=None` (manually triggered), consistent with Otto's event-driven architecture. The Airflow UI provides a built-in Gantt chart, task-level logs, and DAG graph visualization for monitoring pipeline execution and identifying bottlenecks.

Airflow is installed in a separate virtual environment (`airflow-venv`) due to transitive

dependency conflicts with the pipeline's Google Cloud libraries. Full setup instructions are provided in `Data-Pipeline-Guide.md` Section 4.10.

DVC Pipeline

The same four stages are defined in `dvc.yaml` with explicit `deps` and `outs` declarations. Running `dvc repro` executes the full pipeline in dependency order, while DVC's smart caching skips unchanged stages automatically (MD5 hash comparison via `dvc.lock`). Individual stages can also be triggered selectively with `dvc repro <stage>`, which runs any upstream dependencies if needed. DVC handles data versioning — tracking all pipeline artifacts with content-addressable hashing and remote storage on GCS (Section 2.5).

Production Integration

In production, the pipeline stages are orchestrated programmatically by the `/pipeline/run` FastAPI endpoint on Cloud Run — not by Airflow or DVC. However, both the Airflow DAG and DVC pipeline wrap the same production classes (`GitHubIngester`, `EnhancedCodeChunker`, `ChunkEmbedder`) and target the same GCP infrastructure (Cloud Storage, Vertex AI). The `run_pipeline.py` script imports these classes directly with no code duplication, and the target repository is controlled via the `OTTO_REPO` environment variable, making the pipeline reusable for any GitHub repository.

2.5 Data Versioning with DVC

Implementation:
`Data-Pipeline/dvc.yaml`,
`Data-Pipeline/dvc.lock`,
`Data-Pipeline/.dvc/config`

Beyond orchestration (Section 2.4), DVC serves as Otto's data versioning layer, tracking every pipeline artifact with content-addressable hashing and remote storage on GCS.

DVC Remote:

```
gs://otto-pm-processed-chunks/dvc-cache
```

Key Versioning Features:

- **GCS remote storage:** `dvc push` versions all data artifacts to `gs://otto-pm-processed-chunks/dvc-cache`
- **Git integration:** `dvc.lock` is committed to Git, recording exact input/output hashes for every run
- **Reproducibility:** anyone can run `dvc pull` to fetch the exact data used in any previous pipeline run

All pipeline outputs are declared as `outs` in `dvc.yaml`, so DVC tracks them via `dvc.lock` rather than individual `.dvc` files. The `dvc.lock` file records the MD5 hash of every input and output from each run, and is committed to Git alongside `dvc.yaml` and `.dvc/config`, ensuring full reproducibility.

2.6 Tracking and Logging

Implementation: Throughout all pipeline modules

Otto implements comprehensive logging at every stage of the pipeline:

Python Logging:

- All `Data-Pipeline/scripts/` use Python's `logging` module with dual handlers: stdout stream and persistent file (`logs/pipeline.log`)
- Log format: `timestamp [LEVEL] message`
- All errors include full tracebacks via `traceback.print_exc()`

Production Logging (Cloud Run):

- All Cloud Run stdout/stderr is captured to Google Cloud Logging with structured log entries
- Pipeline progress is logged per step with emoji indicators (ingestion file counts, chunk counts, embedding rates, commit SHAs)
- Login-time repository sync logs every repo checked, commit comparison, and update status
- Error alerts trigger with full context: repository name, file path, error type, stack trace

Anomaly Alerts:

- Critical anomalies (missing embeddings, empty content, wrong dimensions).
- Alert format includes repository name and full anomaly list
- Non-critical warnings (outliers, duplicates, tiny chunks) are logged at WARNING level without triggering alerts
- Anomaly alerts are discussed further (Section 2.8)

2.7 Data Schema & Statistics Generation

Implementation: `ingest-service/src/validation/schema_validation.py`

The `SchemaValidator` class validates chunk structure and generates dataset statistics after each pipeline run.

Schema Expectations (6 checks):

Statistics Generated:

- Total chunk count and unique file count
- Language distribution (count per language)
- Chunk size statistics: average, maximum, minimum (in characters)
- Anomaly counts: missing embeddings, missing content, missing language, empty content

Expectation	Rule
required_fields_present	All chunks must have: `content`, `language`, `file_path`, `chunk_index`, `start_line`, `end_line`
content_non_empty	`content` field must be a non-empty, non-whitespace string
language_valid	`language` must be one of 20 supported language values
chunk_index_non_negative	`chunk_index` must be a non-negative integer
start_lte_end_line	`start_line` must be $\leq$ `end_line`
embedding_dim_768	All embeddings must be exactly 768-dimensional (`text-embedding-004`)

Table 2: Schema validation expectations

Output:
`data/processed/schema_validation.json`,
`data/processed/validation_report.json`
These reports contain timestamp, repository name, all expectation results with pass/fail and failed counts, overall pass/fail, and statistics.

2.8 Anomaly Detection & Alerts

Implementation: `ingest-service/src/validation/anomaly_detection.py`

The `AnomalyDetector` class detects data anomalies and triggers alerts. It is automatically called after each pipeline run.

Anomaly Checks (critical — triggers Slack alert):

- **Empty dataset:** zero chunks detected
- **Missing content:** chunks with empty/whitespace content
- **Missing embeddings:** chunks without embedding vectors
- **Missing language:** chunks with no language tag
- **Missing file path:** chunks with no file path
- **Wrong embedding dimension:** embeddings not 768-dimensional

Warning Checks (non-critical — logged only):

- **Outlier chunks:** content size > 3 standard deviations from mean
- **Tiny chunks:** content < 10 characters
- **Huge chunks:** content > 10,000 characters
- **Duplicate chunks:** identical content across chunks
- **High unknown language:** >20% of chunks with undetected language

Alert Mechanism:

- Slack webhook via `SLACK_WEBHOOK_URL` env var
- Alert message format: `:warning: Otto Pipeline Alert \n Repo: {repo} \n {anomaly list}`
- Uses Python's `urllib.request` (no extra dependency)
- Gracefully skips alert if webhook not configured

Output: `data/processed/anomaly_report.json` containing timestamp, total chunks, anomaly list, warning list, pass/fail status, and summary statistics including language distribution and missing value counts.

2.9 Pipeline Flow Optimization

Bottleneck Identification

Pipeline bottlenecks were identified using the Airflow UI's built-in Gantt chart (Section 2.4) and stage-level timing captured in `logs/pipeline.log`. Figure 1 shows the Airflow Gantt chart from a full pipeline run over the Otto repository (220 files, 394 chunks):

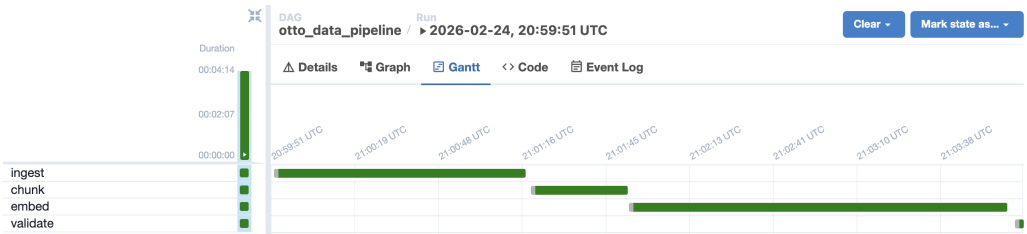

Figure 1: Airflow Gantt chart for a full pipeline run of otto-pm/otto. The embed stage (rightmost bar) dominates total runtime.

The Gantt chart visually confirms that the **embed** stage dominates pipeline execution time, followed by **ingest**. Stage-level timing from `logs/pipeline.log` provides the precise breakdown:

Stage	Duration	Share
Ingest	79s	33.6%
Chunk	27s	11.5%
Embed	127s	54.0%
Validate	<1s	0.3%
Stage Total	**235s**	
Scheduling Overhead	19s	
Wall Clock	**254s**	(4m 14s)

Table 3: Pipeline execution times (otto-pm/otto, 220 files, 394 chunks). Stage durations from `logs/pipeline.log`; wall clock time from Airflow Gantt chart. The 19s overhead reflects Airflow's inter-task scheduling delays.

The **embed** stage is the clear bottleneck at ~54% of stage execution time, driven by Vertex AI API latency. The **ingest** stage follows at ~34%, driven by GitHub API calls.

The 19s of Airflow scheduling overhead (~7.5% of wall clock time) is negligible relative to stage execution and is inherent to Airflow's sequential task dispatch. These findings directly informed the optimizations below.

Embedding Batch Processing

- Vertex AI supports up to 250 texts per batch request
- `ChunkEmbedder` processes chunks in batches of 250, achieving ~216 chunks in 10–15 seconds vs. 3–5 minutes one-by-one
- Throughput: approximately 15–20 chunks/second

Smart Caching

- `CommitTracker` compares current GitHub commit SHA against last processed SHA before any pipeline work begins
- Already-embedded chunks are skipped during re-runs (`force_reembed=False`)
- DVC stage caching: unchanged stages are skipped entirely via MD5 hash comparison in `dvc.lock`

Retry and Resilience

- HTTP retry with exponential backoff on all GitHub API calls
- Individual chunk fallback on batch embedding failure: failed batches retry chunk-by-chunk
- Graceful file skipping: binary files and encoding errors do not halt the pipeline

Planned Architecture Optimization

- Split `ingest-service` into two Cloud Run services: `pipeline-service` (slow, memory-heavy, 4 Gi RAM) and `rag-service` (fast, latency-sensitive, 1 Gi RAM)
- Independent scaling: pipeline scales to 0 when idle, RAG stays warm
- Add Cloud Tasks for async pipeline job queuing

3 Data Bias Detection Using Data Slicing

Implementation: `ingest-service/src/validation/bias_detection.py`

The `BiasDetector` class analyzes embedding coverage consistency across meaningful data slices to detect representation bias in the indexed codebase.

3.1 Detecting Bias in the Data

Since Otto's "demographic features" are code-level attributes rather than human demographics, bias is defined as: *certain categories of code having systematically lower embedding coverage or quality than others*, which would cause RAG retrieval to perform worse on those categories.

The primary metric used is **embedding coverage** (proportion of chunks with embeddings) per slice, as this directly correlates with RAG retrieval quality — chunks without embeddings cannot be retrieved at all.

3.2 Data Slicing for Bias Analysis

Four slicing strategies are implemented:

Slice Type	Groups	Why It Matters
By Language	python, typescript, javascript, java, go, rust, unknown, etc.	Different languages may have worse chunking/embedding support
By File Type	.py, .tsx, .ts, .js, .md, .yaml, .json, etc.	File types may be underrepresented in training data
By Chunk Size	small (<200 chars), medium (200–1000), large (>1000)	Very small or very large chunks may embed poorly
By Repo Section	backend/, frontend/, ingest-service/, docs/, root	Certain directories may be systematically skipped or fail

Table 4: Data slicing strategies for bias detection

Per-slice metrics computed:

- Chunk count per slice
- Embedding coverage (proportion with embeddings)
- Average content size (characters)
- Missing embedding count

Bias detection threshold: A slice is flagged as biased if its embedding coverage is more than 1 standard deviation below the mean coverage across all slices with >5 chunks (to ignore statistically insignificant slices).

3.3 Mitigation of Bias

When biased slices are detected, the following mitigations are applied:

- **Force re-embedding:** The `force_reembed=True` flag on `ChunkEmbedder` clears and regenerates embeddings for all chunks in the affected slice
- **Re-ingestion:** If an entire file type or language is missing, the repository is re-ingested with updated file extension filters
- **Chunker adjustment:** Chunk size parameters (`chunk_size`, `overlap_lines`) can be tuned per language to reduce tiny or oversized chunks
- **Text truncation:** Chunks exceeding 3,072 tokens are truncated before embedding to ensure the model processes the full input

3.4 Bias Mitigation Documentation

The bias detection report (`data/processed/bias_report.json`) records for each slice type:

- Per-slice embedding coverage and chunk counts
- Average coverage across all slices
- List of biased slices (below threshold)
- Specific mitigation recommendation per biased slice
- Overall bias detected flag and global recommendation

Trade-offs: Force re-embedding all chunks in a biased slice increases pipeline runtime (250 chunks/batch $\approx$ 10–15 seconds per batch). For large repositories this may take several minutes. The pipeline accepts this trade-off to ensure equitable retrieval quality across all code types.

4 Additional Guidelines

4.1 Folder Structure

```
otto/
|-- backend/
|   |-- app/
|   |   |-- clients/
|   |   |   +-- ingest_service.py        # Pipeline trigger client
|   |   |-- dependencies/
|   |   |-- models/
|   |   |-- routes/
|   |   |   |-- rag.py                   # RAG proxy to ingest-service
|   |   |   +-- webhook.py               # GitHub webhook handler
|   |   |-- services/
|   |   +-- utils/
|   +-- docs/
|
|-- ingest-service/                      # Core pipeline + RAG service
|   |-- app/
|   |   +-- routes/
|   |       +-- pipeline.py              # Pipeline + RAG endpoints
|   |-- src/
|   |   |-- ingestion/
|   |   |   +-- github_ingester.py
|   |   |-- chunking/
|   |   |   |-- enhanced_chunker.py
|   |   |   |-- chunker.py
|   |   |   +-- embedder.py
|   |   |-- validation/
|   |   |   |-- schema_validation.py
|   |   |   |-- anomaly_detection.py
|   |   |   +-- bias_detection.py
|   |   |-- rag/
|   |   |   |-- rag_services.py
|   |   |   |-- vector_search.py
|   |   |   +-- llm_client_gemini_api.py
|   |   |-- github/
|   |   |   +-- github_client.py
|   |   +-- utils/
|   |       |-- storage_utils.py
|   |       |-- commit_tracker.py
|   |       +-- file_manager.py
|   +-- scripts/
|       |-- ingest_repo.py, process_repo.py
|       |-- embed_repo.py, rag_cli.py
```

```
|       +-- analyze_chunk_quality.py
|
|-- frontend/
|   |-- app/
|   |   |-- api/rag/                    # SSE streaming proxies
|   |   |-- auth/
|   |   +-- project/
|   |-- components/                     # 60+ React components
|   |-- hooks/
|   |-- context/
|   +-- utils/
|
|-- Data-Pipeline/                      # MLOps pipeline (DVC + Airflow)
|   |-- dags/                           # Airflow DAG definition
|   |-- scripts/
|   |   |-- run_pipeline.py             # Airflow/DVC stage runner
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_acquisition.py
|   |   |-- test_preprocessing.py
|   |   +-- test_embedder.py
|   |-- data/
|   |   |-- raw/
|   |   |   +-- metadata.json
|   |   +-- processed/
|   |       |-- chunks.jsonl
|   |       |-- chunks_embedded.jsonl
|   |       |-- schema_validation.json
|   |       |-- anomaly_detection.json
|   |       |-- bias_detection.json
|   |       +-- validation_report.json
|   |-- logs/
|   |   +-- pipeline.log
|   |-- dvc.yaml
|   |-- dvc.lock
|   +-- requirements.txt
|
|-- deliverables/scoping/
|-- style-checker/
+-- README.md, setup-env.sh, setup-env.bat
```

Figure 2: Otto repository structure (pipeline-relevant files)

4.2 README Documentation

Implementation:
`Data-Pipeline-Guide.md` (comprehensive pipeline guide),
`README.md` (repository root, links to the data pipeline guide)

A detailed pipeline guide is provided in `Data-Pipeline-Guide.md` at the repository root. The main `README.md` links to this guide for all data pipeline instructions. The guide covers all four required areas. All sections referenced below are in `Data-Pipeline-Guide.md`:

1. Environment Setup (Sections 4.1–4.3): Step-by-step instructions for cloning the repository, creating a Python 3.11 virtual environment, installing dependencies, authenticating with GCP via a provided service account key (`sa-key.json`), and setting all required environment variables (`GITHUB_TOKEN`, `GCP_PROJECT_ID`, `GCS_BUCKET_RAW`, `GCS_BUCKET_PROCESSED`, `VERTEX_LOCATION`). Platform-specific instructions are provided for macOS/Linux, Windows PowerShell, and Windows CMD.

2. Steps to Run the Pipeline (Sections 3, 4.4–4.7, and 4.10): The guide documents three ways to run the pipeline. Section 3 covers testing the deployed Cloud Run pipeline via `curl` commands for each endpoint (`/pipeline/run`, `/pipeline/ingest`, `/pipeline/chunk`, `/pipeline/embed`), as well as RAG features (Q&A, search, documentation generation, code completion, code editing). Sections 4.4–4.7 cover local execution via DVC, including viewing the DAG with `dvc dag`, running the full pipeline with `dvc repro`, running individual stages, and inspecting outputs. Section 4.10 covers running the Airflow DAG locally, including isolated virtual environment setup, initialization, and triggering via the Airflow UI or CLI.

3. Code Structure Explanation (Section 1.1): A full repository tree diagram showing all pipeline-relevant files, a service responsibility table describing the four top-level services (`backend`, `ingest-service`, `frontend`, `Data-Pipeline`), and an architecture diagram showing how the services connect from user request through to GCS and Vertex AI.

4. Reproducibility and Data Versioning (Sections 4.9–4.10): Explains how `dvc.lock` records MD5 hashes of all pipeline inputs and outputs after each run, how `dvc push`/`dvc pull` enables artifact versioning via the GCS remote (`gs://otto-pm-processed-chunks/dvc-cache`), and includes a table mapping every pipeline artifact to whether it is tracked by Git or DVC (GCS). Section 4.10 documents the Airflow DAG setup, providing an additional orchestration layer with built-in Gantt chart visualization and task-level monitoring.

4.3 Reproducibility

Complete step-by-step instructions for replicating the pipeline on any machine are provided in `Data-Pipeline-Guide.md` (Sections 4.1–4.10). The guide covers cloning the repository, creating a Python 3.11 virtual environment, installing pinned dependencies, authenticating with GCP via a provided service account key, setting environment variables, and running the full pipeline with `dvc repro` or via the Airflow DAG. Platform-specific instructions are included for macOS/Linux, Windows PowerShell, and Windows CMD.

Reproducibility is further supported by DVC's data versioning: `dvc.lock` records MD5 hashes of all inputs and outputs and is committed to Git, providing a verifiable record

of each pipeline run. Versioned artifacts can be pushed to and pulled from the GCS remote via `dvc push`/`dvc pull`. The pipeline can also be executed via the Airflow DAG (`Data-Pipeline-Guide.md` Section 4.10), providing an alternative orchestration path with built-in monitoring and Gantt chart visualization.

4.4 Code Style

- All Python code follows PEP 8 guidelines with a 99-character line limit (explicitly permitted by PEP 8), verified using `pycodestyle` and `autopep8`. To run the script: `pycodestyle` and `autopep8`. To run the script:
  1. Deactivate the current virtual environment
  2. Navigate to `otto/style-checker`
  3. `python -m venv venv`
  4. `source venv/bin/activate`
  5. `pip install -r requirements.txt`
  6. `python style-checker.py ..`
- Modular design: each class has a single responsibility (`GitHubIngester`, `EnhancedCodeChunker`, `ChunkEmbedder`, `SchemaValidator`, `AnomalyDetector`, `BiasDetector`)
- Type hints throughout all function signatures
- Docstrings on all public methods
- Constants defined at module level (`VALID_LANGUAGES`, `REQUIRED_FIELDS`, `PROJECT_ID`, etc.)
- Error handling at every external call with informative log messages and graceful fallbacks

4.5 Error Handling & Logging

- All GitHub API calls wrapped in try/except with retry logic
- File processing errors skip the file and continue (no pipeline halt on single-file failures)
- GCS operations include error logging with full bucket/path context
- Embedding batch failures retry chunk-by-chunk before giving up
- Validation errors log detailed per-expectation failure counts
- All unhandled exceptions in pipeline stages are caught, logged with `traceback.print_exc()`, and re-raised as HTTP 500 responses from the FastAPI endpoints
- Cloud Run logs all stdout/stderr to Google Cloud Logging with structured metadata (revision, project, region)

5 Evaluation Criteria Summary

Criterion	Status	Implementation
Proper Documentation		`README.md`, `Data-Pipeline-Guide.md`, inline comments, and docstrings (Section 4.2)
Modular Syntax & Code		Single-responsibility classes and reusable modules
Pipeline Orchestration (DAG)		Airflow DAG + DVC 4-stage DAG, Airflow Gantt chart (Section 2.4)
Tracking and Logging		Python logging + Google Cloud Logging + Slack alerts
Data Version Control (DVC)		DVC with GCS remote, `dvc.lock` in Git, `dvc push/pull` (Section 2.5)
Pipeline Flow Optimization		Airflow Gantt chart bottleneck analysis, batch embedding, smart caching (Section 2.9)
Schema & Statistics Generation		`SchemaValidator` with 6 expectations
Anomaly Detection & Alerts		`AnomalyDetector` + Slack webhook
Bias Detection & Mitigation		`BiasDetector` with 4 slicing strategies
Test Modules		69/69 pytest tests passing
Reproducibility		`dvc repro`, Airflow DAG, pinned deps, GCP credentials provided, cross-platform README steps (Section 4.3)
README Documentation		Environment setup, pipeline steps, code structure, DVC versioning, Airflow setup (Section 4.2)
Error Handling & Logging		Try/except at all failure points, full tracebacks

Table 5: Evaluation criteria completion status